

[6] Context-Enhanced Relational Operators with Vector Embeddings

저자: Viktor Sanca, Manos Chatzakis, Anastasia Ailamaki (EPFL)

출처: arXiv:2312.01476 (원본 [5] ICDE 2023 비전 논문의 확장 기술 논문)

분량: 약 12 페이지 + 실험 부록

역할군: (A) 이론적 기반

요약

이 논문은 벡터 임베딩을 관계형 데이터베이스 연산에 공식적으로 통합하는 이론적 기초를 제공한다. 핵심 기여는 **임베딩 연산자(E-Operator)**와 **컨텍스트 강화 조인(E-Join)**을 관계형 대수의 정식 연산자로 정의하고, 이를 기반으로 한 최적화 기법들을 제시하는 것이다.

논문의 중심 질문은 "구조화 데이터(숫자, 날짜)와 비구조화 데이터(텍스트, 이미지)를 같은 쿼리 안에서 의미 기반으로 분석하려면 어떻게 해야 하는가?"이다. 기존 SQL은 정확한 일치(exact match)나 범위 비교에만 적합했지만, 임베딩 연산자를 통합하면 "의미적으로 유사한" 데이터를 찾을 수 있게 된다.

핵심 기술 기여:

- 프리페치 최적화:** 각 튜플의 임베딩을 한 번만 계산하여, 모델 호출 비용을 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 감소
- 텐서 조인:** Nested Loop Join을 행렬 곱셈으로 변환하여 약 100 배의 속도 향상 달성
- 인덱스 vs. 스캔 분석:** 선택도에 따라 벡터 인덱스 사용의 효율성이 달라진다는 발견

본 논문과의 관계: Exqutor 는 이 논문의 이론적 기초 위에서, "그렇다면 실행 계획을 세울 때 벡터 연산의 카디널리티를 어떻게 정확히 추정할 것인가?"라는 실용적 문제를 해결한다. 특히 인덱스 조인 vs. 스캔 조인의 선택이 선택도에 따라 달라진다는 이 논문의 발견은, Exqutor 에서 정확한 카디널리티 추정이 왜 중요한지를 직접 입증한다.

상세분석

5.1 해결하고자 하는 문제

현대 데이터 분석에서는 **구조화 데이터**(숫자, 날짜 등)와 **컨텍스트 풍부한 데이터**(텍스트, 이미지, 오디오 등)를 함께 분석해야 한다. 기존 관계형 연산자(SELECT, JOIN 등)는 정확한 일치(exact match)나 범위 비교에만 적합하고, "의미적으로 유사한" 데이터를 찾는 데는 부적합하다.

예를 들어, 두 테이블에서 "비슷한 상품 설명"을 기준으로 조인하고 싶다면:

```
-- 이런 쿼리는 기존 SQL 로 표현 불가
SELECT * FROM table_a JOIN table_b
ON semantic_similarity(a.description, b.description) > 0.8
```

현재는 개발자가:

1. 두 테이블의 텍스트를 추출하고
2. 별도의 ML 파이프라인으로 임베딩을 생성하고
3. 벡터 유사도를 계산하고
4. 결과를 다시 원래 데이터와 합치는

복잡한 수작업을 해야 한다. 이는 데이터 파이프라인을 복잡하게 만들고, 오류가 발생하기 쉽고, 유지보수가 어렵다.

5.2 임베딩 연산자 (E-Operator)

논문은 임베딩을 관계형 대수(Relational Algebra)의 공식 연산자로 정의한다:

$$E\mu(R) = \{t \in R, t \rightarrow \mu(t)\}$$

쉽게 말하면:

- 입력: 관계(테이블) R 과 임베딩 모델 μ
- 출력: R 의 각 튜플(행)에 대해 임베딩 벡터를 추가한 새로운 관계

예를 들어, products 테이블에 BERT 임베딩 모델을 적용하면:

원본 테이블 R:

product_id	name	description
1	노트북	가볍고 휴대하기 편함
2	태블릿	화면이 크고 선명함

임베딩 연산 적용 후:

product_id	name	description	embedding_vector
1	노트북	가볍고 휴대하기 편함	[0.23, -0.12, ..., 0.45]
2	태블릿	화면이 크고 선명함	[0.18, 0.02, ..., 0.51]

이 연산자의 중요한 성질:

1. **역연산 가능:** $E\mu^{-1}(E\mu(R)) = R$ (원래 데이터 복원 가능)
2. **관계형 대수와 호환:** 기존의 관계형 규칙(선택 푸시다운, 조인 재정렬 등)이 그대로 적용
3. **합성 가능:** 다른 관계형 연산과 함께 연속적으로 적용 가능

5.3 컨텍스트 강화 조인 (E-Join)

핵심 기여: 벡터 유사도 기반의 새로운 조인 연산자 정의

$$R \text{ JOIN}_{E,\mu,\theta} S \Leftrightarrow \sigma_{\theta}(E_{\mu}(R) \times E_{\mu}(S)) \Leftrightarrow E_{\mu}(R) \text{ JOIN}_{\theta} E_{\mu}(S)$$

이 수식의 의미:

- 두 테이블 R 과 S 를 각각 임베딩하고
- 임베딩 벡터 간의 유사도(코사인 유사도 등)가 임계값 θ 를 넘는 쌍을 조인 결과로 반환

여러 동치 표현이 있기 때문에, 옵티마이저가 상황에 맞는 가장 효율적인 방식을 선택할 수 있다.

구체적 예:

```
-- 상품 설명이 비슷한 제품들을 찾기
SELECT a.product_id, b.product_id
FROM products a, products b
WHERE embedding_similarity(a.description_vec, b.description_vec) > 0.85
AND a.product_id < b.product_id;
```

이는 다음과 같이 표현될 수 있다:

```
 $\sigma(\text{similarity} > 0.85)(E\_BERT(\text{products\_a}) \text{ JOIN } E\_BERT(\text{products\_b}))$ 
```

5.4 비용 모델과 논리적 최적화

Naive 비용 (최악의 경우)

$$\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + M + C)$$

여기서:

- A = 데이터 접근 비용 (메모리에서 읽기)
- M = 모델(임베딩) 계산 비용 ← **이것이 핵심 병목!**
- C = 유사도 연산 비용 (코사인 유사도 등)

예를 들어 $|R| = 10,000$, $|S| = 10,000$ 인 경우:

- 각 튜플 쌍마다 임베딩을 계산하면: $10,000 \times 10,000 = 1$ 억 번의 모델 호출 필요
- BERT 모델 한 번 호출에 100ms 걸린다면: $1 \text{ 억} \times 100\text{ms} = \text{약 } 1,100 \text{ 일}$ 소요

이는 실무에서 완전히 불가능한 비용이다.

프리페치(Prefetch) 최적화 — 핵심 통찰

$$\text{Cost}(R \text{ JOIN } S) = |R| \times |S| \times (A + C) + (|R| + |S|) \times M$$

혁신적인 발견: 각 튜플의 임베딩은 한 번만 계산하면 된다.

두 테이블을 미리 임베딩해두면, 모델 호출 비용이 $O(|R| \times |S|)$ 에서 $O(|R| + |S|)$ 로 줄어든다.

최적화의 논리:

1. R의 모든 튜플을 임베딩: $M \times |R|$
2. S의 모든 튜플을 임베딩: $M \times |S|$
3. 임베딩 벡터 쌍의 유사도 계산: $|R| \times |S| \times C$ (이미 벡터이므로 빠름)

위의 예에서 이 최적화를 적용하면:

- 모델 호출: $(10,000 + 10,000) \times 100\text{ms} = \text{약 } 33 \text{ 분}$ (1,100 일에서 대폭 감소)

이는 **12 배 이상의 속도 향상**을 가져오며, 실무에서 이 차이는 "불가능한 쿼리 → 실행 가능한 쿼리"의 차이이다.

5.5 물리적 최적화: 텐서 조인 (Tensor Join)

논문의 가장 혁신적인 물리적 최적화는 **Nested Loop Join** 을 **행렬 곱셈으로 변환**하는 것이다.

알고리즘

R의 임베딩을 행렬 **R** ($|R| \times \text{dim}$), S의 임베딩을 행렬 **S** ($\text{dim} \times |S|$)로 구성하면:

$$D = R \times S^T$$

결과 행렬 D의 (i,j) 원소가 R의 i번째 튜플과 S의 j번째 튜플 간의 유사도(코사인 유사도의 경우 정규화된 벡터 간 내적)가 된다.

구체적 예:

```
R = [
  [0.2, -0.1, 0.5], # 상품 A의 임베딩
  [0.3, 0.0, 0.4] # 상품 B의 임베딩
]

S^T = [
  [0.2, 0.3], # 상품 X의 임베딩 (전치)
  [-0.1, 0.0],
  [0.5, 0.4]
]

D = R x S^T = [
  [0.30, 0.35], # A-X: 0.30, A-Y: 0.35
  [0.25, 0.35] # B-X: 0.25, B-Y: 0.35
]
```

왜 이것이 빠른가?

1. **수십 년간 최적화된 라이브러리**: Intel MKL, OpenBLAS, cuBLAS(GPU) 등이 행렬 곱셈을 극도로 최적화했다
2. **SIMD 명령어 활용**: CPU의 AVX-512 같은 명령어로 한 번에 32개 연산을 병렬 처리
3. **캐시 효율성**: 행렬 곱셈의 메모리 접근 패턴은 매우 규칙적이라 캐시 히트율이 높다
4. **데이터 재사용**: 행렬 R의 각 행이 S의 모든 열과 연산되므로, 캐시에 올린 데이터를 최대한 활용

Naive Nested Loop 대비:

```
# Naive 방식
for i in range(len(R)):
    for j in range(len(S)):
        similarity = dot_product(R[i], S[j]) # 1 번씩 계산
        # 메모리 접근이 불규칙하고, 캐시 미스 발생 가능
```

실험 결과

논문은 다양한 크기의 조인에서 성능을 측정했다:

데이터 크기	Naive NLJ	텐서 방식	속도 향상
10k × 10k	7.8 초	1.0 초	7.7 배
100k × 100k	2,100 초	730 초	2.9 배
1M × 1M	timeout	~5 시간	연산 완료

SIMD 적용 시 추가 개선:

- 정규화된 벡터(코사인 유사도 계산 시): 추가 **2 배** 향상
- 전체적으로 Naive 대비 **약 100 배** 속도 향상 가능

5.6 인덱스 조인 vs. 스캔 조인 분석

논문의 또 다른 중요한 발견은 **벡터 인덱스(HNSW)** 사용이 항상 좋은 것은 아니라는 것이다.

선택도에 따른 최적 전략

선택도 범위	최적 방식	이유
0~10%	텐서 스캔	인덱스 탐색 오버헤드 > 전체 스캔 비용
10~30%	HNSW 인덱스	인덱스가 실제로 방문할 벡터를 크게 줄임
30~50%	상황 따라 다름	인덱스의 이점이 감소하기 시작
50%+	텐서 스캔	어차피 대부분의 벡터를 방문해야 하므로 스캔이 효율적

구체적 분석:

낮은 선택도에서 인덱스가 부적절한 이유:

- HNSW 인덱스 탐색에 최소한 몇 ms 가 소요
- 하지만 텐서 스캔은 순차적 메모리 접근이 매우 효율적
- 결과가 1~2%뿐이어도, 인덱스 탐색 오버헤드가 더 클 수 있음

높은 선택도에서 인덱스가 부적절한 이유:

- 결과가 50% 이상이면, 어차피 대부분의 벡터를 방문해야 함
- 이 경우 인덱스의 "건너뛰기" 이점이 사라짐
- 인덱스 메타데이터 접근 오버헤드만 남음

이 분석은 **정확한 선택도 추정**이 실행 계획 선택에 얼마나 중요한지를 보여준다.

5.7 실험 결과 요약

논문은 다양한 데이터셋과 임베딩 모델을 사용해 실험을 수행했다:

데이터셋:

- 텍스트: Wikipedia 초록 (100k 문서)
- 이미지: CIFAR-10 (60k 이미지)
- 합성: 무작위 벡터 (검증용)

임베딩 모델:

- BERT (768 차원)
- ResNet-50 (2048 차원)
- 합성 모델 (64~1024 차원)

주요 결과:

1. 프리페치 최적화: 모델 호출 비용 12 배 감소
2. 텐서 조인: Naive NLJ 대비 최대 100 배 빠름
3. 인덱스 vs. 스캔: 선택도에 따라 최적 방식이 크게 달라짐 (7.7 배 차이)

추가 제기 문제

- 1. 임베딩 모델의 선택이 성능에 미치는 영향:** 논문은 여러 모델을 사용하지만, 모델 크기(파라미터 수)가 실행 시간에 얼마나 영향을 미치는지 더 깊이 분석할 필요가 있다. 특히 대규모 LLM(1B+ 파라미터)을 사용하는 경우의 비용이 어떻게 되는지 명확하지 않다.
- 2. 메모리 제약 환경에서의 성능:** 행렬 곱셈 방식은 메모리 효율적이지만, 메모리가 부족한 환경에서 임베딩 행렬을 저장할 수 없는 경우는 어떻게 처리하는지 언급이 부족하다.
- 3. 동적 데이터 환경:** 실험은 정적 데이터에 대해서만 수행되었다. 새로운 데이터가 계속 삽입되는 환경에서 임베딩을 점진적으로 업데이트하는 방법이 제시되지 않았다.
- 4. 다중 임베딩 필드:** 만약 하나의 테이블에 여러 텍스트 필드가 있고, 각각 다른 임베딩 모델을 사용해야 한다면 비용은 어떻게 증가하는가?
- 5. 신뢰도(Confidence)의 추정:** 현실적으로 임베딩 벡터 간 유사도가 높다고 해서 의미적으로 유사하다는 보장이 완벽하지 않다. 이 불확실성을 옵티마이저가 고려할 수 있도록 확장할 필요가 있다.